

Capstone Project in “Full Stack Apps with AI”

This project assignment for the course “**Full Stack Apps with AI**” in SoftUni AI aims to give you **practical skills** for AI-assisted development of multi-platform full-stack apps with Next.js, React, PostgreSQL and Expo.

Author	Anton Dinkov
Email	a.dinkov@gmail.com
GitHub Repo	https://github.com/antonDinkov/hotel-booking-monorepo.git
Web Project Live URL	https://bookyourstay.netlify.app
Expo Project Live URL	https://bookyourstaymobile.netlify.app
Credentials for testing (login + pass)	Client: peter@abv.bg / 123456 Partner: maya.santoso@nusantarasummit.com / 123456 Admin: admin@abv.bg / 123456 Stripe Test Card Number: 4242 4242 4242 4242 Stripe Info fields: ANY data

* If you have a good reason to keep your codebase or app private, contact the training team for instructions.

1. Project Assignment

Using **agentic AI development** implement and deploy a **fully functional multi-platform full-stack app**.

- Implement a **backend** + **Web client** app + **mobile client** app (for Android / iOS).
- The **topic** and **scope** of your project is up to you.
- Examples: blog system, social network, listings website, recipes catalog, hotel booking platform, issue tracking system, project planner, poll system, e-commerce platform, health tracker, meme generator.

2. Project Requirements

Technologies

- **Backend:** Implement a back-end API with Next.js + PostgreSQL.
- **Database:** Neon serverless PostgreSQL + Drizzle ORM.
- **Web app:** Implement a front-end Web app in Next.js + React + TypeScript + Tailwind.
- **Mobile app:** Implement a client mobile app with React Native + Expo.

Architecture

- Use a **client-server architecture**:
 - React frontend with Next.js backend, communicating via Server Actions.
 - React Native (Expo) mobile client with Next.js backend, communicating via RESTful API.
- Structure your app in a **Node.js monorepo**: Next.js Web app + Expo mobile app.
 - The Next.js app will hold your back-end APIs + Web client app.
 - The Expo app will hold your React Native mobile app.
- **Serverless deployment** on a managed platform (like Netlify) with serverless database (like Neon).

Web App

- Implement **minimum 10 app screens** (pages / popups / others) in the **Web client app**.
 - Examples: register, login, main page, view details page, about page, user dashboard (view / add / edit / delete entity), admin panel.
 - Implement **responsive design** for desktop and mobile browsers.
 - Use **icons, effects** and **visual cues** to enhance user experience and make the UI more intuitive.
- To avoid repeating code, extract repeatable UI code and logic in **reusable components**.

Mobile App

- Implement minimum **5 app screens** (pages / popups / others) in the **mobile app**.
 - Implement **responsive layout** for smartphones (small screens) and tablets (large screens).
 - Implement only **the most important end-user functionality** in the mobile app. Leave the admin panel and other non-essential functionality for the Web app.
- Place different app screens in **separate components** (for easier maintenance).

Backend

- Use **Next.js + Drizzle ORM + PostgreSQL** as a **backend** to keep all app data.
- Store data tables in **PostgreSQL** (e.g. serverless **Neon DB**), accessed with **Drizzle ORM**.
- Use **JWT tokens** for authentication (register, login, logout), **hash passwords** in the DB (e.g. with `'bcrypt'`).
- Use **object storage** (like **Cloudflare R2**) to upload / download **photos** and **files** at the server-side.
- Structure app logic as **services**, exposed through a **RESTful API** and consumed by **Next.js Server Components**.

Authentication and Authorization

- Use **JWT tokens** for **authentication** and **authorization**, with custom code or specialized library like **Auth.js**.
- Implement **users** (register, login, logout) and **roles** (e.g. regular and admin users).
- Store user **passwords** in the DB using secure **one-way hashing** algorithms such as **bcrypt** or **argon2**.
- Enforce **access control** using authorization checks in API endpoints, server-side components, and middleware / interceptor logic.
- Implement **admin panel** in the Web app or similar concept (for special users, different from regular). For example, in a soccer planner app, **group managers** are special users who manage a **group dashboard**.

Database

- Your database should hold **minimum 4 DB tables** (with relationships when needed).
 - Example (blog): users, articles, photos, comments.
 - Example (social network): users, posts, comments, messages.
- Use best practices to **design the DB schema**, including normalization, indexing, and relationships.

- Always use **Drizzle migrations** to make changes in the **DB schema**.
 - Your **DB migration SQL scripts** should be committed in the **GitHub repo**.
- Create a database **seed script** to initially insert sample data in DB.
- Implement your database access logic with **Drizzle API**. Ensure you work efficiently with the DB.

Scalability

- Design the app to **scale efficiently** and maintain **performance** when handling thousands of DB entities.
- Implement **server-side data paging** for data retrieval and display to prevent performance degradation or UI or server unresponsiveness when working with large datasets.
- Populate the primary DB tables with **at least 10,000 records** to validate performance under realistic load conditions. Create **DB indexes** if necessary to speed-up slow DB operations.

Deployment

- Your project should be **deployed live** on the Internet (e.g. to Netlify, Vercel or similar serverless platform).
- Provide **sample credentials** (e.g. demo / demo123) to simplify testing your app.

Requirements for the **Next.js app deployment**:

- Deploy your **Next.js back-end + Web client** as a single project to a hosting platform like **Netlify**.
- Configure your **DATABASE_URL** with your production database in Neon DB.
- Assign a random **JWT_TOKEN** in the production environment.

Requirements for the **Expo mobile app deployment**:

- Deploy your **Expo mobile app** as **Web export** in Netlify or other app hosting platform.
- Connect the backend to your **RESTful API** endpoints from your **Next.js app deployment**.
- Optionally, build an **Android .APK file** (Android binary app image) using Expo EAS, and upload it in your **GitHub Project**, in the "Releases" section.

GitHub Repo

- Use a **GitHub repo** to hold your project assets.
- **Commit and push** each successful change during the development.

Your public **GitHub repo** is the most important project asset for your capstone project! The **commit history** in your repo demonstrates that **you have worked seriously** to develop your app yourself, and you have spent several days working on it. Without a solid history of commits in GitHub, you cannot demonstrate that **your project is your own work** (not taken from someone else).

- Create **minimum 15 commits** in GitHub.
- Create your commits on at least **3 different days**.
- You can commit directly or you can work in branches and merge them with pull requests.

AI Agent Instructions

- Define **agent instructions** file **AGENTS.md** to describe the app context, architectural guidelines, technologies and patterns and other project-wide instructions for the AI dev agent.
- The agent's instructions file is not documentation. Instead, it **helps AI agents to make correct decisions** when they work with the code. Instructions like "*to modify the DB always use Drizzle migrations*" help the agents work correctly and maintain the project structure, architecture and code conventions.

Documentation

- Generate a **project documentation** in your GitHub repository.
- **Project description**: describe briefly your project (what it does, who can do what, etc.).
- **Architecture**: front-end, back-end, database, APIs, mobile app, technologies, components and communication between them.
- **Database schema** design: visualize the main DB tables and their relationships.
- **Repo structure**: key **folders**, **files** and their purpose.
- Local **development setup** guide: how to clone and run your app.

File Storage

- Store app **user files** (like photos and documents) in an **object storage** system (like Cloudflare R2).
- Implementing **file uploads** and **downloads** is a **non-mandatory requirement**.

Automates Tests

- Optionally, implement **automated tests** for the app functionality, to maintain high quality of avoid regressions
- Automate the test execution in **GitHub Actions**.
- Implementing automates tests is a **non-mandatory requirement**.

Automated Backups

- Optionally, implement automated **database / file storage backups**:
 - Implement the backup script in **GitHub Actions** (provides free, reliable, short-running VMs).
 - Backup the PostgreSQL DB as **compressed DB dump** and the storage buckets as **zip** archive.
 - Keep backup archives in a **private object storage** bucket (e.g. in Cloudflare R2).
 - Implement a backup **retention policy**: 7 daily, 5 weekly, 12 monthly backups.
 - Run the backup script **once daily**, e.g. at 3:00 AM.
- Implementing **DB / file backups** is a non-mandatory requirement.

3. Project Implementation

Develop your project using modern **AI-assisted development**, with AI dev agents like **GitHub Copilot**, **Claude Code**, **Codex**, **Roo Code**, **Cursor**, **Amazon Q**.

Split the project into **manageable steps** and use the traditional **AI dev loop**:

- Prompt the agent → implement → run → test → refine → commit and push to GitHub / discard changes

Keep **full change history** in GitHub (commits, pull requests, etc.)

4. Project Assessment

Criteria	Score	Description	My Score
GitHub Commits	0...15	At least 15 commits (1 score per commit, max 15)	...
GitHub Commit Days	0...15	Commits on 3 different days (5 score per day, max 15)	...
Architecture	0...5	Project structure (monorepo, Next.js app, Expo app), client-server architecture with HTTP communication	...

Backend	0...5	Backend services and API endpoints , with DB persistence and auth system	...
Database	0...5	At least 4 DB tables (2 score per table, max 8), with Drizzle ORM and migrations	...
Users and Roles	0...5	Implemented correctly users, roles , login, register, logout, authentication and authorization	
Scalability	0...5	Paging in the APIs and UI for large tables, tested with large datasets	
Web App	0...15	Implemented at least 10 app screens , with responsive design	...
Admin Panel	0...5	Implement users, roles and admin panel (or special users dashboard / panel)	...
Mobile App	0...15	Implement a client mobile app, connecting to the server API, with at least 5 mobile app screens	...
Deployment	0...5	Projects (Next.js app and Expo app) are published live on the Internet and work properly	...
Documentation	0...5	Project documentation in the GitHub repo	...
File Uploads (Bonus)	-	Files / photos uploads / downloads from object storage	...
Automated Tests (Bonus)	-	Automated tests for the apps (back-end, Web client, mobile app): unit tests, integration tests, end-to-end tests	...
Backups (Bonus)	-	Automated DB / file storage backups	...